Com S 227
Spring 2026
Assignment 1
200 points

Due Date: Friday, March 13, 11:59 pm (midnight)
5% bonus for submitting 1 day early (by 11:59 pm March 12)
10% penalty for submitting late (by 11:59 pm March 15)

This assignment is to be done on your own. See the Academic Integrity policy in the syllabus, for details.

You will not be able to submit your work unless you have completed the *Academic Dishonesty policy questionnaire* on the Assignments page on Canvas. Please do this right away.

If you need help, try office hours, the schedule is on Canvas. Lots of help is also available through the Piazza discussions. Please start the assignment as soon as possible to get your questions answered right away. It is physically impossible for the staff to provide individual help to everyone the afternoon that the assignment is due!

This is a "regular" assignment so we are going to read your code. Your score will be based partly (about a two thirds) on functional tests and partly on the grader's assessment of the quality of your code. See the "More about grading" section.

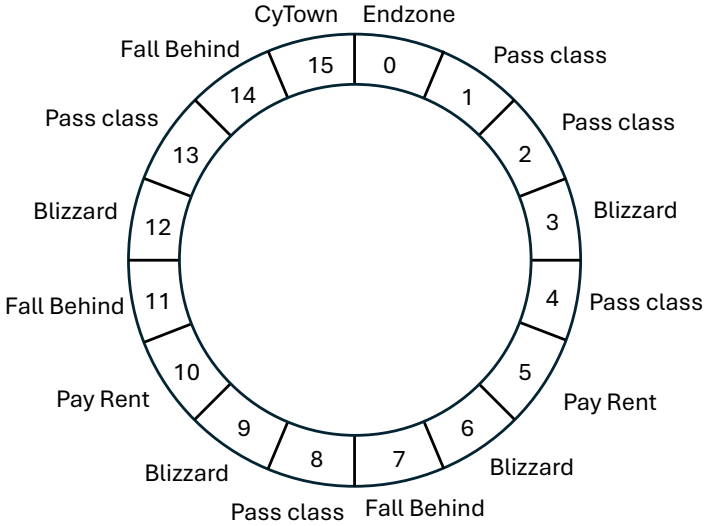
Contents

Overview 2
Specification 3
Where's the main() method?? 3
Suggestions for getting started 3
Gradescope ..... Error! Bookmark not defined.
More about grading 5
Style and documentation 6
If you have questions 6
What to turn in 7

Overview

The purpose of this assignment is to give you lots of practice working with conditional logic and managing the internal state of a class. You'll create a single class, called **`CyGame`**, that models a Monopoly-like game. The game is based around the idea of two players taking turns rolling dice and each moving a game piece over the squares on a board. The type of square a piece lands on has a set of rules (for example, one type of square gives a player an extra turn). Each player also has a pile of game money and can use that money to buy property. The game ends when one of the players has at least **`MONEY_TO_WIN`** money or one of the players goes bankrupt (has negative money).

Here is how the game is setup. The board consists of one or more squares which are arranged sequentially and wrap around forming a circular pattern. The squares are numbered starting at 0 and ending at one less than the number of squares. For consistence, in the figures below we will always draw the board in clockwise order.



Each square has exactly one type. Square type is determined using the following rules, applied in strict precedence order (only the first matching rule applies):

1. **Endzone** is always on the first square (i.e., square 0).
2. **CyTown** is always on the last square (i.e., the one that comes before square 0).
3. Every 5th square (e.g., 5, 10, 15...) is **Pay Rent**.
4. Every 7th and every 11th square is **Fall Behind**.
5. Every 3rd square is **Blizzard**.
6. All remaining squares are **Pass Class**.

Order matters, for example, square 15 is Pay Rent even though 15 is divisible by 3.

Both players start with their game piece on square 0. They take turns rolling a 6-sided die (i.e., rolls 1 to 6). After each roll they move their piece forward (clockwise) on the board by the same number of squares as the value of the roll. An exception to this rule is when a player is currently on a Blizzard type square, in that case they can only move forward if the die roll is an odd

number. The type of square landed on determines some action the play must perform. The actions are:

- *Endzone*. When a player lands on or passes over the endzone they are awarded an endzone prize money (**`ENDZONE_PRIZE`**). A player cannot both land on the square and pass over it (i.e., player are only awarded the endzone prize each time they pass around the board. A player can only collect the prize once per turn (i.e., passing the endzone multiple times in a turn doesn't result in extra payouts).
- *CyTown*. The CyTown square is a property. When a player lands on this square, if it is not already owned, they have the option (by calling the **`buyCyTown()`** method) of buying the square for **`CYTOWN_COST`** amount of money. The owner of the CyTown square charges higher rents (see Pay Rent square below).
- *Pay Rent*. When a player lands on the pay rent square they must pay the other player. The standard cost of rent is the constant **`STANDARD_RENT_PAYMENT`**. However, if the other player owns CyTown then the cost or rent is doubled (i.e., 2 x **`STANDARD_RENT_PAYMENT`**).
- *Fall Behind*. The player goes backward one square and performs whatever action that square requires. However, if the new square is also a square that requires moving the player forward or backward then the turn ends on that square (i.e., the player cannot fall behind twice or fall behind and pass a class in the same turn). This prevents the need to use loops to implement this logic.
- *Blizzard*. No additional action is taken for the remainder of the players turn. In future turns, the player stays stuck on the blizzard square until they roll an odd number.
- *Pass Class*. The player moves forward an additional 4 squares and performs whatever action that square requires. However, if the new square is also a square that requires moving the player forward or backward then the turn ends on that square (i.e., the player can not pass two classes or pass a class and fall behind in the same turn). This prevents the need to use loops to implement this logic. If the player passes over the endzone square when moving forward apply the usual rule of paying the player the **`ENDZONE_PRIZE.`**

The game ends when one of two conditions are meet 1) one of the players has reached a winning amount of money (**`MONEY_TO_WIN`**) or 2) one of the players has gone bankrupt (they have a negative balance of money).

Specification

The specification for this assignment includes this pdf, the online Javadoc, and any "official" clarifications announced on Canvas.

Where's the main() method??

There isn't one! Like most Java classes, this isn't a complete program and you can't "run" it by itself. It's just a single class, that is, the definition for a type of object that might be part of a

larger system. To try out your class, you can write a test class with a main method like the examples below in the getting started section.

Gradescope (see below) will perform a lot of functional tests, but when you are developing and debugging your code at first you'll always want to have some simple test cases of your own (some example simple tests are provided to you in a separate file), as in the getting started section below.

Suggestions for getting started

Smart developers don't try to write all the code and then try to find dozens of errors all at once; they work ***incrementally*** *and test every new feature as it's written. Since this is our first assignment, here is an example of some incremental steps you could take in writing this class.*

1. Create a new, empty project and then add a package called **`hw1`**. ***Be sure to choose "Don't Create" at the dialog that asks whether you want to create module-info.java***.

2. Create the **`CyGame`** class in the **`hw1`** package. You can copy the code from the posted skeleton that has the constants and the string methods. Add stubs for all the methods and constructors described in the Javadoc. Document each one. For methods that need to return a value, just return a "dummy" value as a placeholder. At this point there should be no compile errors in the project.

3. Start with the constructor **`CyGame()`** and the method **`getCurrentPlayer()`**. The presence of a getter suggests the need for an instance variable to keep track of the current player. It will need to be initialized in the constructor. Try a very simple test like this:

```
public class CyGameTest {
    public static void main(String args[]) {
        CyGame game = new CyGame(16, 200);
        System.out.println("Expected start is Player 1*:
(0, false, $200) Player 2: (0, false, $200)");
        System.out.println(game);
        System.out.println();
    }
}
```

You should see an output string like this, produced by the **`toString()`** method:

```
Player 1*: (0, false, $0) Player 2: (0, false, $0)
```

Note that in printing the variable **`game`**, the **`println`** method automatically invokes the method **`game.toString()`**. The **`toString()`** method is calling **`getCurrentPlayer()`**.

4. Next, each square is assigned a type, now would be a good time to complete the **`getSquareType()`** method because it will be difficult to test any of the mutator methods without having square types. The rules for determining the square type are clearly described in the javadoc provided with the assignment files. The figure in the Overview Section above shows how square types should be assigned for a 16 square board. Try a simple test.

```
System.out.println("Board squares: ");
System.out.println(game.getSquareType(0) + " expect 0");
System.out.println(game.getSquareType(1) + " expect 5");
System.out.println(game.getSquareType(2) + " expect 5");
System.out.println(game.getSquareType(3) + " expect 4");
System.out.println(game.getSquareType(4) + " expect 5");
System.out.println(game.getSquareType(5) + " expect 2");
System.out.println(game.getSquareType(6) + " expect 4");
System.out.println(game.getSquareType(7) + " expect 3");
System.out.println(game.getSquareType(8) + " expect 5");
System.out.println(game.getSquareType(9) + " expect 4");
System.out.println(game.getSquareType(10) + " expect 2");
System.out.println(game.getSquareType(11) + " expect 3");
System.out.println(game.getSquareType(12) + " expect 4");
System.out.println(game.getSquareType(13) + " expect 5");
System.out.println(game.getSquareType(14) + " expect 3");
System.out.println(game.getSquareType(15) + " expect 1");
System.out.println();
```

7. From here you can continue using the provided file CyGameTest.java to guide implementing the methods.

11. At some point, upload your code to Gradescope. *Always start reading error messages from the top.* If you have a missing or extra public method, if the method names or declarations are incorrect, or if something is really wrong like the class having the incorrect name or package, any such errors will appear *first* in the output and will usually say "Class does not conform to specification." **Always fix these first.**

More about grading

This is a "regular" assignment so we are going to read your code. Your score will be based partly (about 2/3) on Gradescope's functional tests and partly on the TA's assessment of the quality of your code. This means you can get partial credit even if you have errors, but it also means that **even if you pass all the Gradescope tests you can still lose points**. Are you doing things in a simple and direct way that makes sense? Are you defining redundant instance variables? Are you using a conditional statement when you could just be using **`Math.min`**? Are you using a loop for something that can be done with integer division? Some specific criteria that are important for this assignment are:

- Use instance variables only for the “permanent” state of the object, use local variables for temporary calculations within methods.
    - You will lose points for having unnecessary instance variables
    - All instance variables should be **`private`**.
- **Accessor methods should not modify instance variables**.

See the "Style and documentation" section below for additional guidelines.

Style and documentation

Roughly 15% of the points will be for documentation and code style. Here are some general requirements and guidelines:

- **Each class, method, constructor and instance variable, whether public or private, must have a meaningful javadoc comment**. The javadoc for the class itself can be very brief, but must include the **`@author`** tag. The javadoc for methods must include **`@param`** and **`@return`** tags as appropriate.
    - Try to briefly state what each method does in your own words. However, there is no rule against copying the descriptions from the online documentation. *However: **do not literally copy and paste from this pdf**! This leads to all kinds of weird bugs due to the potential for sophisticated document formats like Word and pdf to contain invisible characters.*
    - Run the javadoc tool and see what your documentation looks like! (You do not have to turn in the generated html, but at least it provides some satisfaction :)
- All variable names must be meaningful (i.e., named for the value they store).
- Your code should **not** be producing console output. You may add **`println`** statements when debugging, but you need to remove them before submitting the code.
- Internal (//-style) comments are normally used inside of method bodies to explain *how* something works, while the Javadoc comments explain *what* a method does. (A good rule of thumb is: if you had to think for a few minutes to figure out how something works, you should probably include an internal comment explaining how it works.)
    - Internal comments always *precede* the code they describe and are indented to the same level.
- Use a consistent style for indentation and formatting.
    - Note that you can set up Eclipse with the formatting style you prefer and then use Ctrl-Shift-F to format your code. To play with the formatting preferences, go to Window->Preferences->Java->Code Style->Formatter and click the New button to create your own “profile” for formatting.

Gradescope

A significant portion (but not all) of the points for this assignment are based on the gradescope autograder tests. It is not advised to wait until the last day to run the gradescope on your code.

You have an unlimited number of submissions, and unless we receive complaints from gradescope, we will not restrict the rate at which you can submit.

Do not try to "cheat" the tests by "hardcoding" your code to pass specific tests. You are expected to implement a solution that meets the specifications described in this document, the tests are only provided to gain confidence that your solution is correct. We reserve the right to add new "hidden" tests during grading and the graders will look for "hardcoding" during the manual part of the grading.

You can find the autograder for this Assignment on gradescope. Please see the last section of this document for more information on how to submit to Gradescope.

If you have questions

For questions, please see the Piazza Q&A pages and click on the folder **`hw1`**. If you don't find your question answered, then create a new post with your question. Try to state the question or topic clearly in the title of your post, and attach the tag **`hw1`**. *But remember, do not post any source code for the classes that are to be turned in.* It is fine to post source code for general Java examples that are not being turned in. (In the Piazza editor, use the button labeled "pre" to have Java code formatted the way you typed it.)

If you have a question that absolutely cannot be asked without showing part of your source code, make the post "private" so that only the instructors and TAs can see it. Be sure you have stated a specific question; vague requests of the form "read all my code and tell me what's wrong with it" will generally be ignored.

Of course, the instructors and TAs are always available to help you. See the Office Hours section of the syllabus to find a time that is convenient for you. We do our best to answer every question carefully, short of actually writing your code for you, but it would be unfair for the staff to fully review your assignment in detail before it is turned in.

Any announcements from the instructors on Canvas that are labeled "Official Clarification" are considered to be part of the spec, and you may lose points if you ignore them. Such posts will

always be placed in the Announcements section of Canvas. (We promise that no official clarifications will be posted within 24 hours of the due date.)

What to turn in

Note: You will need to complete the "Academic Dishonesty policy questionnaire," found on the Assignments page on Canvas, before the submission link will be visible to you.

Assignments are turned in by Gradescope, there is nothing to submit on Canvas. The assignment will be called "Assignment 1" on Gradescope. It will run a number of functional tests and report

how many (and which) tests your code passed. There are many test cases so there may be an overwhelming number of error messages. ***Always start reading the errors at the top and make incremental corrections in the code to fix them. Please submit just the file called CyGame.java, and at the top, there should be a line that puts it in the hw1 package, that is, "package hw1;"***. Do not create or submit any zipfiles with Gradescope even though gradescope may offer you the chance to submit a zipfile. Please note that any 5% bonus for early submission or 10% penalty for late submission will not be calculated by Gradescope itself. Instead, it will be calculated by a separate script later. So you will only see your raw score once the Teaching Assistants have graded your work.